

Question 2: modify the algorithm to find minimum forest on an u connected graph G and count the number of trees

```
#include <vector>
#include <list>
#include <queue>

typedef int WeightType;

struct Edge {
    int u, v;
    WeightType w;
};

struct Graph {
    Graph(int n): E(n) {}
    std::vector<std::list<Edge>> E;
    void add_edge(int u, int v, WeightType w) {
        E[u].push_back({u, v, w});
        E[v].push_back({v, u, w});
    }
    int n() const { return E.size(); }
    int m() const {
        int res = 0;
        for (auto& list : E)
            res += list.size();
        return res / 2;
    }
};

//since in a disconnected graph, we can't visit all vertices starting form one single starting node.
// we run the prim's algorithm once for each unvisited component
//we also use a counter to count the number of trees we process.
// I modify the MST to return a forest and count the number of trees
std::pair<Graph, int> MSF(const Graph &G) {
    auto WeightComp = [](const Edge e1, const Edge e2) { return (e1.w > e2.w); };
    std::priority_queue<Edge, std::vector<Edge>, decltype(WeightComp)>
candidates(WeightComp);

    Graph forest(G.n());
    std::vector<bool> visited(G.n(), false);
    int num_trees = 0;

    for (int start = 0; start < G.n(); ++start) {
        if (visited[start]) continue;

        // New tree
        num_trees++;
        visited[start] = true;

        for (Edge e : G.E[start])
            candidates.push(e);

        while (true) {
            while (!candidates.empty()) {
                Edge temp = candidates.top();
                if (visited[temp.u] && visited[temp.v])
                    candidates.pop();
            }
        }
    }
}
```

```

        else
            break;
    }
    if (candidates.empty())
        break;

    Edge safe = candidates.top();
    candidates.pop();
    int u = visited[safe.v] ? safe.u : safe.v;
    visited[u] = true;
    forest.add_edge(safe.u, safe.v, safe.w);

    for (Edge e : G.E[u]) {
        if (!visited[e.v])
            candidates.push(e);
    }
}
}
return {forest, num_trees};
}

```

Question 3. Please design a function to detect an edge is useless or not and analyze the time complexity of Kruskal's algorithm which running with your useless checker.

> Hint: Use [Disjoint set](https://en.wikipedia.org/wiki/Disjoint-set_data_structure) may make your algorithm efficient.

```

struct DisjointSet {
    std::vector<int> parent, rank;

    DisjointSet(int n): parent(n), rank(n, 0) {
        for (int i = 0; i < n; ++i) parent[i] = i;
    }

    int find(int u) {
        if (parent[u] != u)
            parent[u] = find(parent[u]); // Path compression
        return parent[u];
    }

    bool unite(int u, int v) {
        int ru = find(u);
        int rv = find(v);
        if (ru == rv) return false; // Edge is useless (creates a cycle)
        if (rank[ru] < rank[rv])
            parent[ru] = rv;
        else {
            parent[rv] = ru;
            if (rank[ru] == rank[rv])
                rank[ru]++;
        }
        return true; // Edge is safe
    }
};

```

```

Graph KruskalMST(const Graph& G) {
    std::vector<Edge> edges;

```

```

for (int u = 0; u < G.n(); ++u)
    for (Edge e : G.E[u])
        if (u < e.v) // to avoid duplicates
            edges.push_back(e);

std::sort(edges.begin(), edges.end(), [](Edge a, Edge b) {
    return a.w < b.w;
});

DisjointSet ds(G.n());
Graph T(G.n());

for (Edge e : edges) {
    if (ds.unite(e.u, e.v)) {
        T.add_edge(e.u, e.v, e.w);
    }
}
return T;
}

```

Question 4

```
int get_path_cost(const Graph &T, int v) {
    std::vector<bool> visited(T.n(), false);
    std::vector<int> cost(T.n(), 0); // cost[i] = cost from root to i

    std::function<void(int)> dfs = [&](int u) {
        visited[u] = true;
        for (const Edge &e : T.E[u]) {
            int neighbor = e.v;
            if (!visited[neighbor]) {
                cost[neighbor] = cost[u] + e.w;
                dfs(neighbor);
            }
        }
    };

    dfs(0); // start from root

    return cost[v]; // cost from root to node v
}
```